Workbench Line Display — Demo Guide

A plain-language guide for running and demoing the multi-monitor assembly line display. No programming needed to run it. Read this top to bottom once; the demo itself is just **run one file and start scanning.**

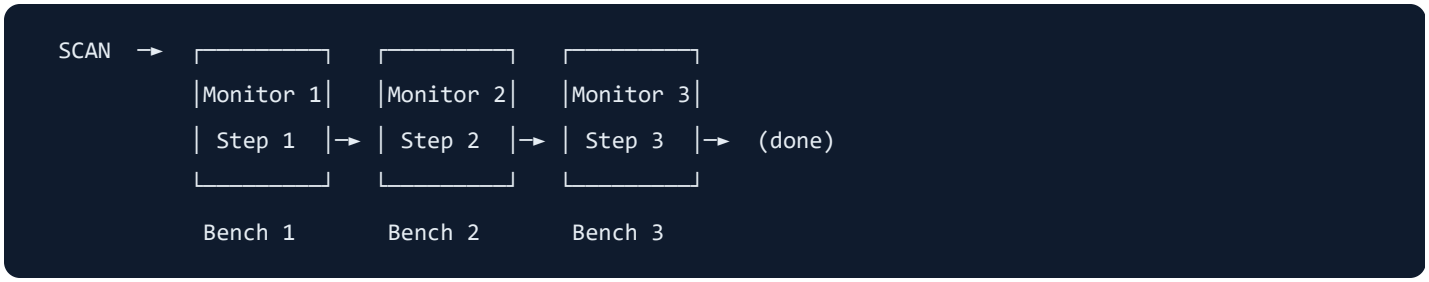
---

1. What this software does (the 30-second version)

There is **one barcode scanner** at the start of the assembly line and **several TV monitors**, one above each workbench.

- A worker scans a part → **Monitor 1** shows **Step 1** for that part.
- The part moves to the next bench. The worker scans the **next** part → the new part shows **Step 1 on Monitor 1**, and the first part automatically **slides over to Monitor 2, now showing Step 2.**
- Keep scanning, and each part keeps sliding down the line, one monitor at a time, showing the next step at each bench.

Think of it like a **conveyor belt of instructions**: every scan pushes everything one monitor forward. Workers never touch a keyboard or mouse — they just scan.



There are also two helper web pages for you (not for the workers):

- **Supervisor page** — one screen showing the whole line at a glance.
- **Simulator page** — lets you "fake" scans by clicking buttons, so you can rehearse the demo **without** the real scanner.

---

2. What you need (hardware)

Item	Notes
1 Windows PC	Windows 10 or 11. This runs everything.
1 barcode scanner	Zebra DS3678 (the one from the plant), in **serial / COM-port mode**.
Monitors / TVs	One per bench. For the demo, even 1–2 screens is fine — the software adapts to however many you configure.
Cables/adapters	However the monitors plug into the PC (HDMI, USB display adapters, etc.). Windows must already "see" all monitors in **Settings → System → Display**.
Google Chrome	Must be installed. The displays run as full-screen Chrome windows.

> **You do not need internet for the demo.** Everything runs locally on the one PC.

---

3. First-time setup (do this once, before demo day)

You need two files in the program folder that are **not** in the download because they get built from the source code:

- `workbench-line-display.exe` — the main program (the screens)
- `serial-capture.exe` — the scanner reader (only if using the real scanner)

You have two ways to get them:

Option A — Ask Visvesh for the ready-made `.exe` files. Easiest. Just drop the `.exe` files into the program folder next to `config.json`. Done.

Option B — Build them yourself (needs Go installed, free from https://go.dev/dl/). Just **double-click** `build.bat`. It builds all the `.exe` files for you and tells you when it's done. (The first time, it needs internet to download some code.) You only do this once, or again if the code changes.

> If you prefer the manual way, open a terminal in the program folder and run `go build -o workbench-line-display.exe .` then `go build -o serial-capture.exe ./serial-capture`.

---

4. How to run it (the actual demo)

The easy way — one file

Double-click `start.bat`.

It does everything automatically:

1. Starts the display program.
2. Starts the scanner reader.
3. Opens one full-screen Chrome window per monitor.

Then just **scan parts** and watch the screens.

> To close a full-screen Chrome window, press **Alt + F4**. To stop the programs, close the small black command windows that opened.

Rehearsing without the real scanner

Open a normal Chrome window and go to:

```
http://localhost:8080/simulator
```

You'll see a button for each part. **Click a part button** = same as scanning it. Great for practicing the flow at your desk before the client arrives.

Useful web addresses (type into Chrome)

Address	What it shows
`http://localhost:8080/`	Home page with links to everything
`http://localhost:8080/supervisor`	**Whole-line overview** — put this on your laptop to narrate the demo
`http://localhost:8080/simulator`	**Fake scans** for rehearsing
`http://localhost:8080/display/D1`	What Bench 1's monitor shows (D2, D3… for the others)

---

5. Getting the monitors on the right screens

`start.bat` opens each Chrome window at a screen position. If a window opens on the wrong monitor, you fix it by editing two numbers.

1. Right-click `start.bat` → **Edit** (opens in Notepad).
2. Find the lines near the bottom that look like this:

```
start "" %CHROME% %FLAGS% --user-data-dir="...D1" --window-position=1920,0   .../display/D1
start "" %CHROME% %FLAGS% --user-data-dir="...D2" --window-position=3000,0   .../display/D2
start "" %CHROME% %FLAGS% --user-data-dir="...D3" --window-position=4080,0   .../display/D3
```

3. The `--window-position=X,Y` is the **top-left corner** of that monitor.
   - `X` = how far from the left (in pixels), `Y` = how far down.
   - The **main** monitor (plant's own software) starts at `0,0` and is **not** listed here — these are the extra bench monitors.
   - Example: if each monitor is 1920 wide, the monitors to the right of the main one start at `1920`, `3840`, `5760`, and so on.
4. Save, close, and run `start.bat` again.

> Tip: you can drag a window to the right monitor by hand and skip this. The positions in `start.bat` just save you doing it every time.

---

6. Configuring it — `config.json`

Everything the demo person might change lives in **one file:** `config.json`, in the program folder. Open it with Notepad.

> ⚠️ **It must stay valid.** Keep every `"quote"`, `,` comma and `{ }` bracket exactly as they are. If the program won't start after an edit, you almost certainly removed or added a comma. Make a copy of the file before editing so you can undo.

Here is the whole file with plain-English notes:

```
{
  "code_pattern": "P(\\d{5})",                          // how to read the scan - leave as is
  "server_port": 8080,                                  // web address port - leave as is

  "serial_port": "COM11",                               // which COM port the scanner reader listens on
  "serial_mode": "baud=9600 parity=N data=8 stop=1",    // scanner speed - must match the Zebra's settings
  "serial_mirror_port": "",                             // advanced - leave blank
  "serial_debug": false,                                // set true only when troubleshooting the scanner

  "displays": [                                         // <<< THE NUMBER OF MONITORS
    { "id": "D1", "name": "Bench 1 — Step 1" },
    { "id": "D2", "name": "Bench 2 — Step 2" },
    { "id": "D3", "name": "Bench 3 — Step 3" }
  ],

  "parts": {                                            // <<< THE PARTS AND THEIR STEPS
    "82301": {
      "name": "82301GB000MQC · MQ4I Front Door Trim (FL)",
      "steps": [
        { "image": "assets/MQ4I-FRONT-step1.svg", "description": "Mount the bracket and seat all clips" },
        { "image": "assets/MQ4I-FRONT-step2.svg", "description": "Fit inside handle, screw to garnish (x3)
        { "image": "assets/MQ4I-FRONT-step3.svg", "description": "Map pocket + mood lamp screwing" }
      ]
    }
    // ... more parts ...
  }
}
```

6a. Change the NUMBER OF MONITORS

Edit the `displays` list. One line per monitor, **in line order** (Bench 1 first).

- **To add a 4th monitor:** add a line (notice the comma after the previous line):

```
"displays": [
  { "id": "D1", "name": "Bench 1 — Step 1" },
  { "id": "D2", "name": "Bench 2 — Step 2" },
  { "id": "D3", "name": "Bench 3 — Step 3" },
  { "id": "D4", "name": "Bench 4 — Step 4" }
],
```

- **To use fewer monitors:** delete a line (and fix the comma on the line above so the last one has no trailing comma).
- `id` = the short code in the web address (`D1` → `http://localhost:8080/display/D1`). Keep them unique.
- `name` = the heading that appears on that screen.
- **After adding/removing a monitor, also add/remove a matching Chrome line in** `start.bat` (see Section 5).

6b. Change the PARTS or their STEPS

Each entry under `parts` is keyed by the **part number** (the 5 digits the scanner reads, e.g. `82301`).

- `name` — the big label shown on screen. Put whatever is human-readable.
- `steps` — the list of steps, **in order**. Step 1 shows on Bench 1, Step 2 on Bench 2, and so on.
  - `description` — the instruction text for that step.
  - `image` — the picture for that step. It must be a file inside the `assets` folder, and the name here must match the file name **exactly** (capital letters matter).

To add a new part, copy an existing block, change the number, name, and steps. Example:

```
  "84501": {
    "name": "84501 · New Part Name",
    "steps": [
      { "image": "assets/newpart-step1.svg", "description": "First thing to do" },
      { "image": "assets/newpart-step2.svg", "description": "Second thing to do" }
    ]
  }
```

> A part can have **fewer steps than there are monitors** — once it runs out of steps, its later benches just stay blank. That's normal, not an error.

6c. Replacing the step pictures with real photos

Right now the step pictures are **placeholders**. When the real work-standard images are ready:

1. Put the image files into the `assets` folder.
2. In `config.json`, change each step's `"image"` to the new file name.
3. Restart (`start.bat`). Nothing else changes.

---

7. The scanner (only for the real-scanner demo)

- The Zebra scanner must be in **serial / COM port mode** and set to **9600 baud, 8 data bits, No parity, 1 stop bit** (that's what `serial_mode` says).
- `serial_port` in `config.json` must be the **COM port number our reader listens on.** If the plant's own software is also reading the scanner, a "COM port splitter" (com0com) makes two copies of the port — their software reads one, ours reads the other. In that setup, `serial_port` is **our** copy (e.g. `COM11`), not the real one.
- To check which COM ports exist: Windows **Device Manager** → **Ports (COM & LPT)**.

If you are NOT using the real scanner for the demo, ignore this whole section and use the Simulator page (Section 4) instead.

---

8. Part numbers you can scan in the demo

These four are configured and ready:

Scan this part	What it is
82301	MQ4I Front Door Trim (Front-Left)
82302	MQ4I Front Door Trim (Front-Right)
83301	MQ4I Rear Door Trim (Rear-Left)
83302	MQ4I Rear Door Trim (Rear-Right)

Scan any of them in a row to watch the line fill up and slide forward.

---

9. Suggested demo script (what to say and do)

1. **Before the client arrives:** run `start.bat`, confirm each monitor shows its "waiting" screen, and open the **Supervisor** page on your laptop.
2. "Each bench has a monitor. The worker only ever scans — no keyboard, no mouse."
3. **Scan part 82301.** → "Bench 1 immediately shows Step 1 for this part."
4. **Scan part 82302.** → "The new part takes Bench 1's Step 1, and the first part has automatically moved to Bench 2, now showing Step 2 — the instructions follow the part down the line."
5. **Scan 83301**, then **83302**. → "Three benches busy, each showing the correct step for the part in front of it. The oldest part has finished and left the line."
6. Show the **Supervisor** page: "From here a line leader sees every bench at once, and can reset the line with one button."
7. "Adding more benches or more parts is just editing one text file — no new software."

---

10. If something goes wrong

Problem	Fix
A monitor shows a black screen / "site can't be reached."	The display program didn't start, or started after Chrome. Close everything and run `start.bat` again. Make sure `workbench-line-display.exe` is in the folder.
Program won't start / black window flashes and closes.	`config.json` has a typo (usually a missing or extra comma). Restore your backup copy, or re-check Section 6.
A step shows a broken-image icon.	The `image` name in `config.json` doesn't match a file in the `assets` folder (check spelling and capital letters), or the file is missing.
Scanning does nothing (real scanner).	Wrong `serial_port`, or the plant's software grabbed the port. Confirm the COM number in Device Manager, confirm `serial_mode` matches the scanner, and set `"serial_debug": true` to log what's coming in. Meanwhile you can demo with the Simulator.
Chrome opens on the wrong monitor.	Drag it to the right one, or set `--window-position` in `start.bat` (Section 5).
I made a mess of `config.json`.	That's why you made a backup copy. Replace the file with the backup and restart.

---

11. One-line summary to hand over

> First time only: double-click `build.bat` (or get the `.exe` files from Visvesh). Then plug in the monitors, run `start.bat`, and scan. To change the number of screens or the parts/steps, edit `config.json`. To rehearse without a scanner, open `http://localhost:8080/simulator`.